

Progetto Basi di Dati

Sistema di Gestione Integrata ISP

Giuseppe Angelo Mazzara

Matricola: 1000045472

Anno Accademico 2025/2026

2 maggio 2026

Indice

1	Progettazione Concettuale	3
1.1	Specifica dei Requisiti	3
1.2	Glossario dei Termini	4
1.3	Operazioni sui dati	4
1.4	Sviluppo dello Schema Concettuale	6
1.4.1	Fase 1: Identificazione delle Entità e Macro-Aree Logiche	6
1.4.2	Fase 2: Definizione delle Cardinalità	6
1.4.3	Fase 3: Mappatura degli Attributi e Identificatori	7
1.4.4	Fase 4: Schema Definitivo con Gerarchia ISA e Attributi Derivati	7
1.5	Dizionario dei Dati	9
1.6	Dizionario delle Relazioni	9
1.7	Vincoli non esprimibili e dati derivabili	9
1.8	Trasformazione della gerarchia nel modello logico	11
2	Progettazione Logica	12
2.1	Tavola dei volumi	12
2.2	Tavola delle frequenze	12
2.3	Schema Logico	12
2.4	Implementazione e Generazione dello Schema EER	14
3	Progettazione Fisica	16
3.1	Analisi degli Indici e Scelte Prese	16
3.1.1	Confronto dei costi di ricerca	16
3.1.2	Analisi dell'indice su IP_POOL (idx_ip_address)	16
3.1.3	Analisi dell'indice su FATTURA (idx_fattura_stato_data)	17
3.2	Implementazione	17
3.2.1	Valutazione dell'overhead di scrittura	18
3.3	Normalizzazione (3NF e BCNF)	18
3.3.1	Terza Forma Normale (3NF)	18
3.3.2	Forma Normale di Boyce-Codd (BCNF)	19
3.4	DDL completo	20
3.5	Implementazione della Logica Attiva (Trigger)	24
3.6	Query SQL per le operazioni principali	29
3.7	Dump del database popolato con dati significativi	32
4	Utilizzo dell'Intelligenza Artificiale	34

1 Progettazione Concettuale

1.1 Specifica dei Requisiti

Dati di carattere generale:

Si vuole progettare il sistema informativo di un Internet Service Provider (ISP). Il sistema gestisce l'erogazione di servizi di connettività, la manutenzione della rete e il ciclo di fatturazione.

Dati sui clienti e contratti:

Per i clienti (circa 500.000) memorizziamo nome o ragione sociale, codice fiscale o Partita IVA, email e indirizzo. I clienti si dividono in persone fisiche e giuridiche. Ogni cliente può sottoscrivere diversi **Contratti**, caratterizzati da una data di stipula, uno stato (attivo, sospeso, cessato) e la tecnologia usata (FTTH, ADSL, FWA). Ogni contratto è legato a un **Piano Tariffario** che definisce il canone mensile e la velocità.

Dati sull'infrastruttura di rete:

La rete è composta da **Nodi** identificati da una posizione GPS e una capacità. Ogni nodo gestisce un **IP Pool** (indirizzi statici o dinamici). Ad ogni contratto è associato un **Apparato** hardware (modem/antenna) identificato dal MAC Address e dalla versione firmware.

Dati sulle sessioni e traffico:

Il sistema registra i **Log di Sessione** degli utenti (circa 2 milioni al giorno). Per ogni sessione memorizziamo l'inizio, la fine, l'IP assegnato e il volume di traffico (byte trasmessi e ricevuti). La storia degli ultimi 12 mesi deve essere mantenuta.

Dati su ticket e personale tecnico:

Il personale è composto da **Tecnici** (circa 1.000) identificati da un codice, con specializzazione e costo orario. In caso di guasto si aprono dei **Ticket di Assistenza** con descrizione e priorità. Per risolvere un ticket si effettuano uno o più **Interventi Tecnici** di cui registriamo data, ore lavorate ed esito.

Dati su fatture e pagamenti:

Il sistema rilascia **Fatture** mensili per i contratti. Ogni fattura ha un totale e uno stato (saldo o meno). Per ogni fattura possono esserci diversi **Pagamenti** (transazioni parziali o totali) di cui tracciamo data, importo e metodo di pagamento.

1.2 Glossario dei Termini

Entità	Descrizione	Collegamenti
Cliente	Anagrafica (fisica/giuridica) che usufruisce dei servizi.	Contratto, Fattura
Contratto	Accordo tecnico per una linea specifica.	Piano, Apparato, Nodo
Piano_Tariffario	Offerta commerciale (costi e prestazioni).	Contratto
Nodo_Rete	Punto fisico di smistamento della rete.	IP Pool, Contratto
Apparato	Dispositivo hardware (Modem/ONT) fornito al cliente.	Contratto
IP_Pool	Insieme di indirizzi di rete gestiti dal nodo.	Nodo, Sessione
Sessione_Log	Dati di connessione e volumi di traffico.	Contratto, IP Pool
Ticket	Richiesta di supporto per un guasto.	Contratto, Intervento
Tecnico	Dipendente addetto alle riparazioni fisiche.	Intervento
Intervento	Attività pratica svolta per risolvere un ticket.	Ticket, Tecnico
Fattura	Documento fiscale per il pagamento del servizio.	Contratto, Pagamento
Pagamento	Singola transazione economica effettuata dal cliente.	Fattura

1.3 Operazioni sui dati

1. **O1:** Inserimento nuovo contratto con assegnazione apparato e IP.
2. **O2:** Registrazione massiva log di sessione (alta frequenza).
3. **O3:** Identificazione cliente tramite IP e timestamp.
4. **O4:** Registrazione intervento tecnico e chiusura ticket.
5. **O5:** Generazione fatture mensili per contratti attivi.
6. **O6:** Registrazione pagamento e aggiornamento stato fattura.
7. **O7:** Sospensione contratti con fatture insolute da 60 giorni.

Operazioni Automatiche e Vincoli Attivi (Trigger)

A completamento delle operazioni manuali e massive, il sistema implementa una logica attiva avanzata per garantire l'integrità dei dati e l'automazione dei processi di business dell'ISP:

- **T1 - Protezione Infrastruttura (BEFORE DELETE su Nodo_Rete):** Impedisce l'eliminazione accidentale di un nodo dalla rete qualora vi siano ancora contratti attivi o pool di IP ad esso strettamente associati.
- **T2 - Chiusura Automatica Ticket (AFTER INSERT su Intervento):** Nel momento in cui un tecnico registra un intervento sul campo con "Esito = Positivo", il sistema aggiorna automaticamente lo stato del Ticket padre in "Risolto".
- **T3 - Riconciliazione Contabile (AFTER INSERT su Pagamento):** Ad ogni nuovo pagamento parziale o totale, ricalcola dinamicamente il saldo versato; se la somma eguaglia il totale della fattura, quest'ultima transita automaticamente nello stato "Saldato".
- **T4 - Blocco Clienti Morosi (BEFORE INSERT su Contratto):** Intercetta e blocca l'inserimento di un nuovo contratto se l'anagrafica del *Cliente* risulta già associata a fatture insolute pregresse da oltre 90 giorni.
- **T5 - Unicità Apparato Fisico (BEFORE INSERT/UPDATE su Contratto):** Controlla che il *MAC_Address* dell'Apparato in dotazione non sia già correntemente in uso da un altro contratto attivo, prevenendo accessi abusivi o cloni di rete.
- **T6 - Prevenzione Conflitti IP (BEFORE INSERT su Sessione_Log):** Garantisce che lo stesso Indirizzo IP estratto dall'*IP_Pool* non venga assegnato a due connessioni contemporaneamente attive, evitando sovrapposizioni di routing.
- **T7 - Validazione Temporale Logica (BEFORE INSERT/UPDATE su Contratto):** Applica un vincolo di coerenza cronologica, impedendo a livello di motore database che la *Data_Activazione* o la *Data_Cessazione* possano essere inserite con un valore antecedente alla *Data_Stipula*.

1.4 Sviluppo dello Schema Concettuale

La progettazione concettuale del database è stata condotta attraverso un processo iterativo. Di seguito vengono documentate le quattro fasi principali dello sviluppo del diagramma Entità-Relazione (E/R).

1.4.1 Fase 1: Identificazione delle Entità e Macro-Aree Logiche

In questa prima fase esplorativa, il dominio applicativo è stato strutturato definendo lo scheletro del sistema. Le entità sono state raggruppate in tre macro-aree funzionali per garantire una forte coesione logica: l'**Area Commerciale**, l'**Area Infrastruttura** e l'**Area Servizi**. L'entità *Contratto* è stata posizionata come fulcro centrale.

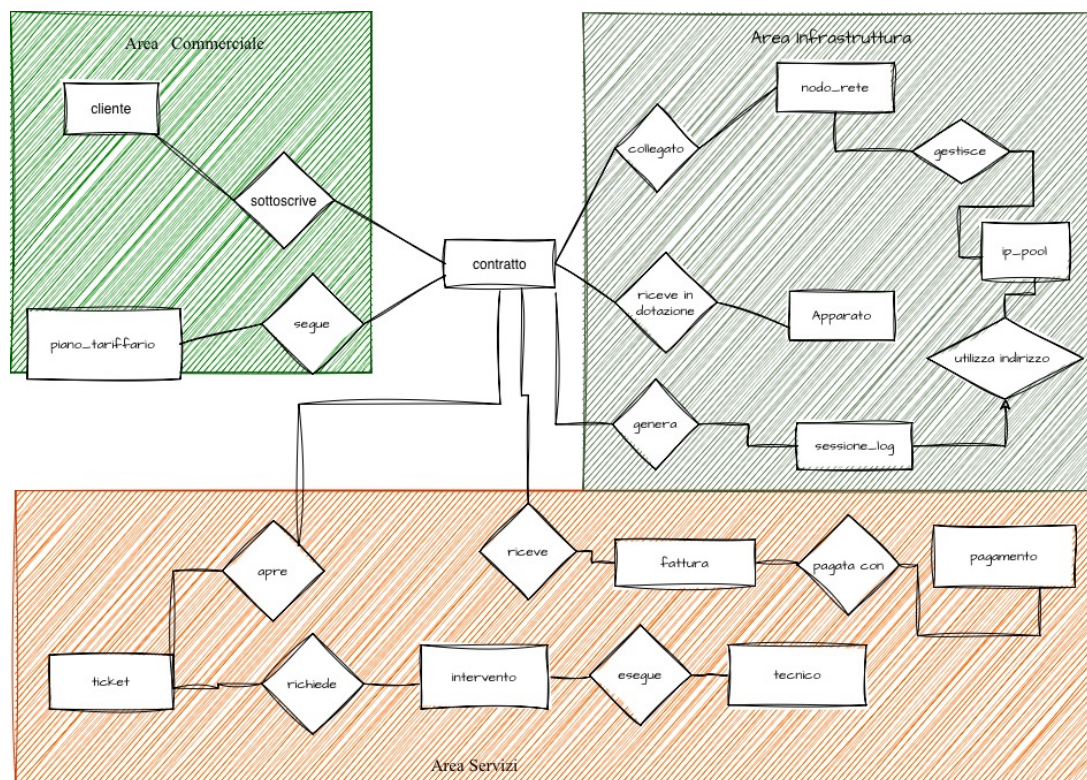


Figura 1: Bozza scheletrica iniziale con suddivisione in macro-aree logiche.

1.4.2 Fase 2: Definizione delle Cardinalità

Il diagramma è stato affinato introducendo i vincoli di partecipazione per ciascuna associazione, definendo le cardinalità minime e massime (*min*, *max*) per mappare accuratamente la semantica aziendale.

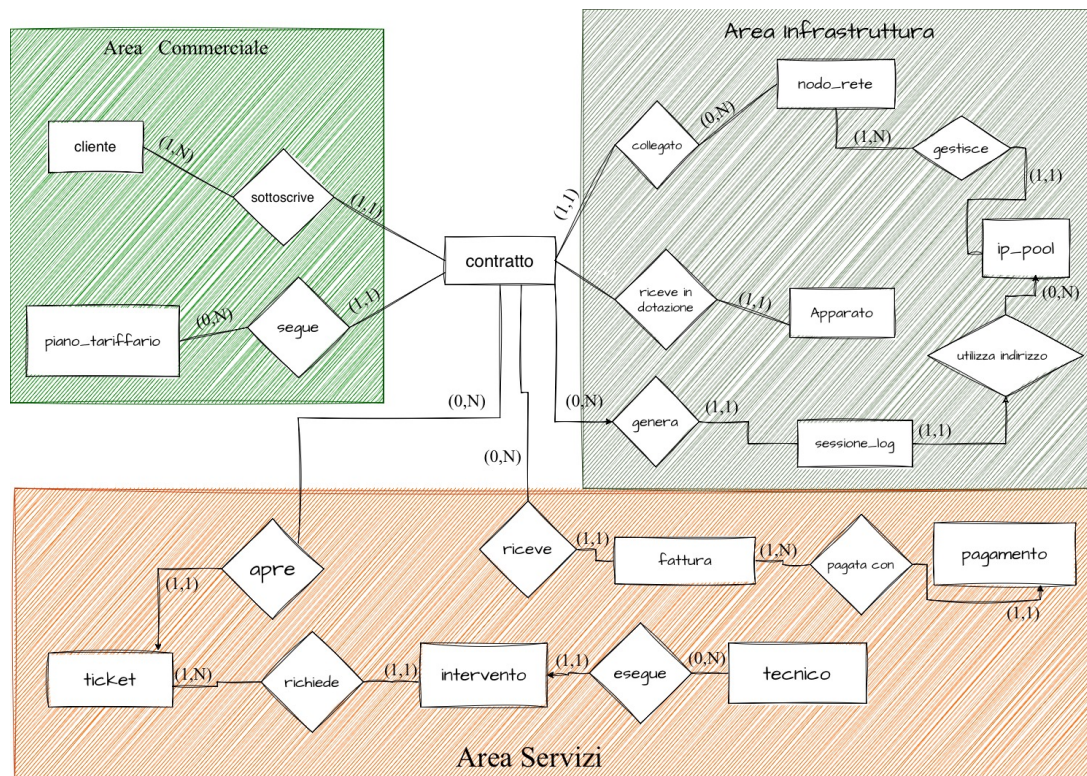


Figura 2: Schema concettuale intermedio con l'introduzione delle cardinalità.

1.4.3 Fase 3: Mappatura degli Attributi e Identificatori

Utilizzando la notazione accademica a "pallini", sono stati inseriti tutti gli attributi nativi per ciascuna entità. Particolare attenzione è stata rivolta all'individuazione delle **Chiavi Primarie** (pallino pieno nero), fondamentali per garantire l'univocità.

1.4.4 Fase 4: Schema Definitivo con Gerarchia ISA e Attributi Derivati

È stata implementata una **Gerarchia di Generalizzazione (ISA)** sull'entità *Cliente*, specializzandola in *Privato* e *Azienda*. Inoltre, sono stati inseriti gli **Attributi Derivati** (es. totale fattura, linea tratteggiata), per indicare valori computati dinamicamente dal sistema.

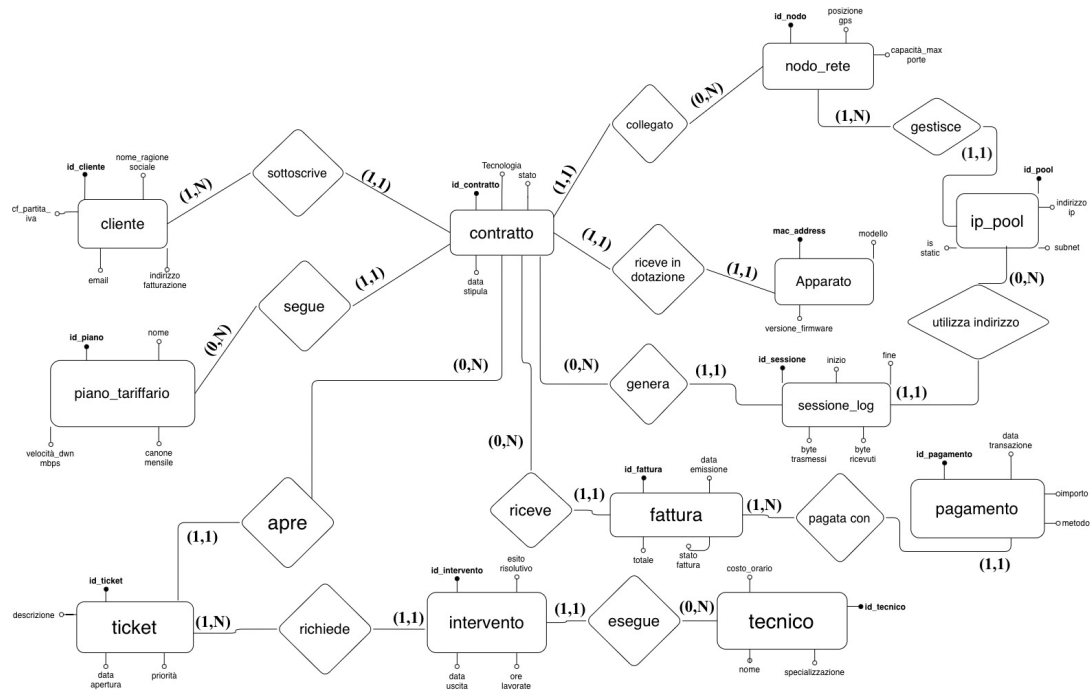


Figura 3: Integrazione degli attributi descrittivi e delle chiavi primarie.

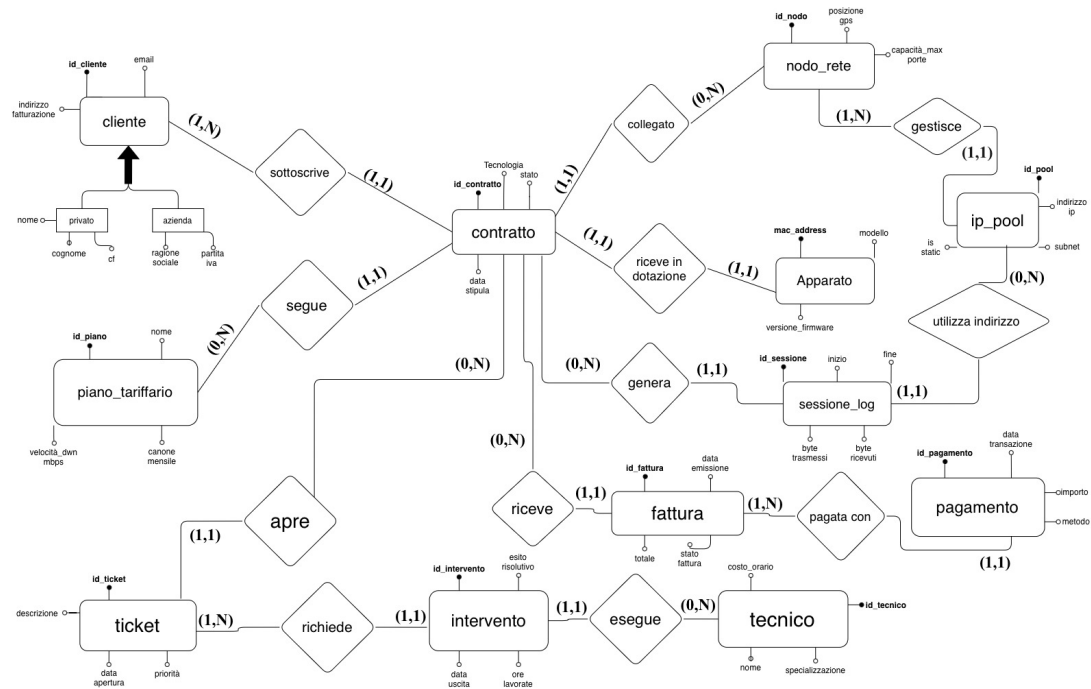


Figura 4: Schema E/R definitivo completo di gerarchie ISA e attributi derivati.

1.5 Dizionario dei Dati

Entità	Descrizione	Attributi
Cliente	Anagrafica base	<u>ID_Cliente</u> , Email, Indirizzo_Fatturazione
Privato	Specializzazione Cliente	Nome, Cognome, CF
Azienda	Specializzazione Cliente	Ragione_Sociale, Partita_IVA
Contratto	Linea attiva dell'utente	<u>ID_Contratto</u> , Data_Stipula, Stato, Tecnologia
Piano_Tariffario	Offerta sottoscritta	<u>ID_Piano</u> , Nome, Canone_Mensile, Velocità
Nodo_Rete	Centrale fisica	<u>ID_Nodo</u> , Posizione_GPS, Capacità_Porte
Apparato	Router fornito	<u>MAC_Address</u> , Modello, Versione_Firmware
IP_Pool	Rete logica del nodo	<u>ID_Pool</u> , Indirizzo_IP, Subnet, Is_Static
Sessione_Log	Dati navigazione	<u>ID_Sessione</u> , Inizio, Fine, Byte_TX, Byte_RX
Ticket	Segnalazione guasto	<u>ID_Ticket</u> , Data_Apertura, Descrizione, Priorità
Tecnico	Personale operativo	<u>ID_Tecnico</u> , Nome, Specializzazione, Costo_Orario
Intervento	Lavoro sul campo	<u>ID_Intervento</u> , Data_Uscita, Ore_Lavorate, Esito
Fattura	Documento fiscale	<u>ID_Fattura</u> , Data_Emissione, Totale, Stato
Pagamento	Transazione	<u>ID_Pagamento</u> , Data, Importo, Metodo

1.6 Dizionario delle Relazioni

1.7 Vincoli non esprimibili e dati derivabili

Nello schema E/R sono presenti dei dati derivabili, mantenuti come ridondanze per motivi di performance (evitare join e calcoli complessi a runtime):

- **Totale Fattura:** Derivabile matematicamente sommando il *Canone_Mensile* del *Piano_Tariffario* e gli eventuali costi orari derivanti dagli *Interventi* tecnici. Viene

Relazione	Entità Coinvolte	Descrizione e Cardinalità
Sottoscrive	Cliente, Contratto	Un cliente sottoscrive (1,N) contratti. Il contratto appartiene a (1,1) cliente.
Segue	Piano, Contratto	Un piano ha (0,N) contratti. Il contratto segue (1,1) piano.
Collegato a	Contratto, Nodo	Un contratto è collegato a (1,1) nodo. Il nodo serve (0,N) contratti.
Riceve in dota.	Contratto, Apparato	Relazione 1 a 1: un contratto ha (1,1) apparato esclusivo.
Gestisce	Nodo, IP_Pool	Un nodo gestisce (1,N) pool. Il pool è di (1,1) nodo.
Genera	Contratto, Sessione	Un contratto genera (0,N) log. Il log è di (1,1) contratto.
Utilizza indir.	Sessione, IP_Pool	Una sessione usa (1,1) indirizzo dal pool.
Apri	Contratto, Ticket	Un contratto apre (0,N) ticket. Il ticket è per (1,1) contratto.
Richiede	Ticket, Intervento	Un ticket richiede (1,N) interventi. L'intervento risolve (1,1) ticket.
Esegue	Tecnico, Intervento	Un tecnico esegue (0,N) interventi. L'intervento è di (1,1) tecnico.
Riceve	Contratto, Fattura	Un contratto riceve (0,N) fatture nel tempo.
Pagata con	Fattura, Pagamento	Una fattura è saldata da (1,N) pagamenti (es. acconti).

calcolato e storicizzato al momento dell'emissione della fattura.

Inoltre, vi sono vincoli di integrità applicativi che non possono essere descritti dalla grafica dello schema e andranno gestiti tramite Trigger lato DBMS:

- **Vincolo di Rete (Integrità Nodi):** Impossibilità di rimuovere fisicamente un *Nodo_Rete* se questo gestisce ancora contratti con stato "Attivo".
- **Integrità dei Pagamenti:** Quando la somma degli importi nella tabella *Pagamento* raggiunge o supera il *Totale* della *Fattura*, lo stato della fattura deve commutare automaticamente su "Saldata".
- **Chiusura Automatica Ticket:** Se un *Intervento* viene registrato con "Esito = Positivo", il *Ticket* padre deve passare allo stato "Chiuso/Risolto".

1.8 Trasformazione della gerarchia nel modello logico

Nello schema concettuale è presente una generalizzazione (gerarchia ISA) sull'entità **Cliente**, suddivisa nei sotto-tipi **Privato** e **Azienda**. Tale gerarchia è *totale* (un cliente è per forza o privato o azienda) ed *esclusiva* (non può essere entrambi).

Per la trasformazione verso il modello relazionale, si è optato per il **collassamento verso l'alto (Accorpamento nel padre)**. Verrà generata un'unica tabella logica **CLIENTE** contenente tutti gli attributi del padre e dei figli, unificando le chiavi. Questa scelta è motivata dai seguenti vantaggi:

1. **Performance:** Evita l'utilizzo di JOIN costose ogni volta che si deve leggere l'anagrafica completa partendo da un Contratto.
2. **Integrità Referenziale Semplificata:** L'entità *Contratto* dovrà puntare esclusivamente a un'unica tabella madre, senza dover gestire FOREIGN KEY condizionali su due tabelle figlie distinte.

Per distinguere le due tipologie di clienti all'interno della tabella unica, in fase logica verrà introdotto un attributo discriminante **Tipo_Cliente** (es. char 'P' o 'A'), e gli attributi non pertinenti al tipo (es. Partita IVA per un privato) verranno ammessi come valori NULL.

2 Progettazione Logica

2.1 Tavola dei volumi

I volumi stimati per il sistema a regime, ipotizzando 1 anno di attività per un Internet Service Provider (ISP) di medie-grandi dimensioni (circa 500.000 clienti attivi), sono i seguenti:

Concetto	Tipo	Volume (Record Stimati)
Cliente	Entità	500.000
Contratto	Entità	600.000
Piano_Tariffario	Entità	25
Nodo_Rete	Entità	1.500
Apparato	Entità	600.000
IP_Pool	Entità	5.000
Sessione_Log	Entità	730.000.000
Ticket	Entità	182.500
Tecnico	Entità	1.000
Intervento	Entità	200.000
Fattura	Entità	7.200.000
Pagamento	Entità	7.500.000

Tabella 1: Tavola dei volumi stimati per 1 anno di attività dell'ISP.

2.2 Tavola delle frequenze

Frequenza stimata delle operazioni sul database a regime, tenendo conto dell'elevato carico di rete e dei processi di billing massivi:

2.3 Schema Logico

Dallo schema concettuale è stato derivato lo schema logico relazionale. Seguendo le scelte progettuali, la gerarchia sull'entità *Cliente* è stata risolta tramite il collassamento nel padre, introducendo l'attributo discriminante **Tipo_Cliente** e ammettendo valori nulli per i campi non comuni. Lo schema non presenta relazioni multi-a-molti (N:M), pertanto non è stato necessario creare tabelle ponte.

Operazione	Tipo	Frequenza
O1 - Inserimento nuovo contratto	I	500 volte / giorno
O2 - Registrazione massiva log di sessione	I	2.000.000 volte / giorno
O3 - Identificazione legale tramite IP	L	15 volte / giorno
O4 - Registrazione intervento e chiusura ticket	I/U	550 volte / giorno
O5 - Generazione fatture mensili	I	1 volta / mese (Massiva)
O6 - Registrazione pagamento e check saldo	I/U	20.000 volte / giorno
O7 - Sospensione contratti morosi (60 gg)	L/U	1 volta / giorno (Batch)

Tabella 2: Tavola delle frequenze (I = Inserimento, L = Lettura, U = Aggiornamento).

Di seguito le relazioni definitive del database.

Legenda: Le chiavi primarie (Primary Key) sono sottolineate con linea continua, mentre le chiavi esterne (Foreign Key) sono evidenziate tramite sottolineatura tratteggiata.

- **CLIENTE** (ID_Cliente, Email, Indirizzo_Fatturazione, Tipo_Cliente, Nome, Cognome, CF, Ragione_Sociale, Partita_IVA)
- **PIANO_TARIFFARIO** (ID_Piano, Nome, Canone_Mensile, Velocita)
- **NODO_RETE** (ID_Nodo, Posizione_GPS, Capacita_Porte)
- **APPARATO** (MAC_Address, Modello, Versione_Firmware)
- **IP_POOL** (ID_Pool, Indirizzo_IP, Subnet, Is_Static, ID_Nodo)
- **CONTRATTO** (ID_Contratto, Data_Stipula, Stato, Tecnologia, ID_Cliente, ID_Piano, ID_Nodo, MAC_Address)
- **SESSIONE_LOG** (ID_Sessione, Inizio, Fine, Byte_TX, Byte_RX, ID_Contratto, ID_Pool)
- **TICKET** (ID_Ticket, Data_Apertura, Descrizione, Priorita, ID_Contratto)
- **TECNICO** (ID_Tecnico, Nome, Specializzazione, Costo_Orario)
- **INTERVENTO** (ID_Intervento, Data_Uscita, Ore_Lavorate, Esito, ID_Ticket, ID_Tecnico)

- **FATTURA** (ID_Fattura, Data_Emissione, Totale, Stato, ID_Contratto)
- **PAGAMENTO** (ID_Pagamento, Data, Importo, Metodo, ID_Fattura)

2.4 Implementazione e Generazione dello Schema EER

Per validare la correttezza della progettazione logica e garantire l'assenza di errori strutturali prima della messa in produzione, l'intero modello relazionale è stato implementato utilizzando il *Data Definition Language* (DDL) del DBMS **MySQL**.

Il codice SQL è stato progettato rispettando rigorosamente l'ordine di creazione delle tabelle, partendo dalle entità forti (indipendenti) fino ad arrivare alle entità deboli (dipendenti), garantendo così la corretta imposizione di tutti i vincoli di integrità referenziale (**FOREIGN KEY**) e definendo policy specifiche per la gestione delle anomalie (**ON DELETE CASCADE** e **ON DELETE RESTRICT**).

Una volta eseguito lo script di creazione sul server locale, è stato impiegato lo strumento **CASE MySQL Workbench**. Attraverso la funzionalità di *Reverse Engineering*, il tool si è interfacciato con il database appena istanziato, analizzando i metadati delle tabelle e le relazioni fisiche esistenti.

Questo processo automatizzato ha permesso di generare il **Diagramma EER** (Enhanced Entity-Relationship) definitivo, riportato nella figura sottostante. L'uso di un tool professionale certifica l'allineamento perfetto tra l'astrazione logica progettata e la reale implementazione fisica sul motore del database.

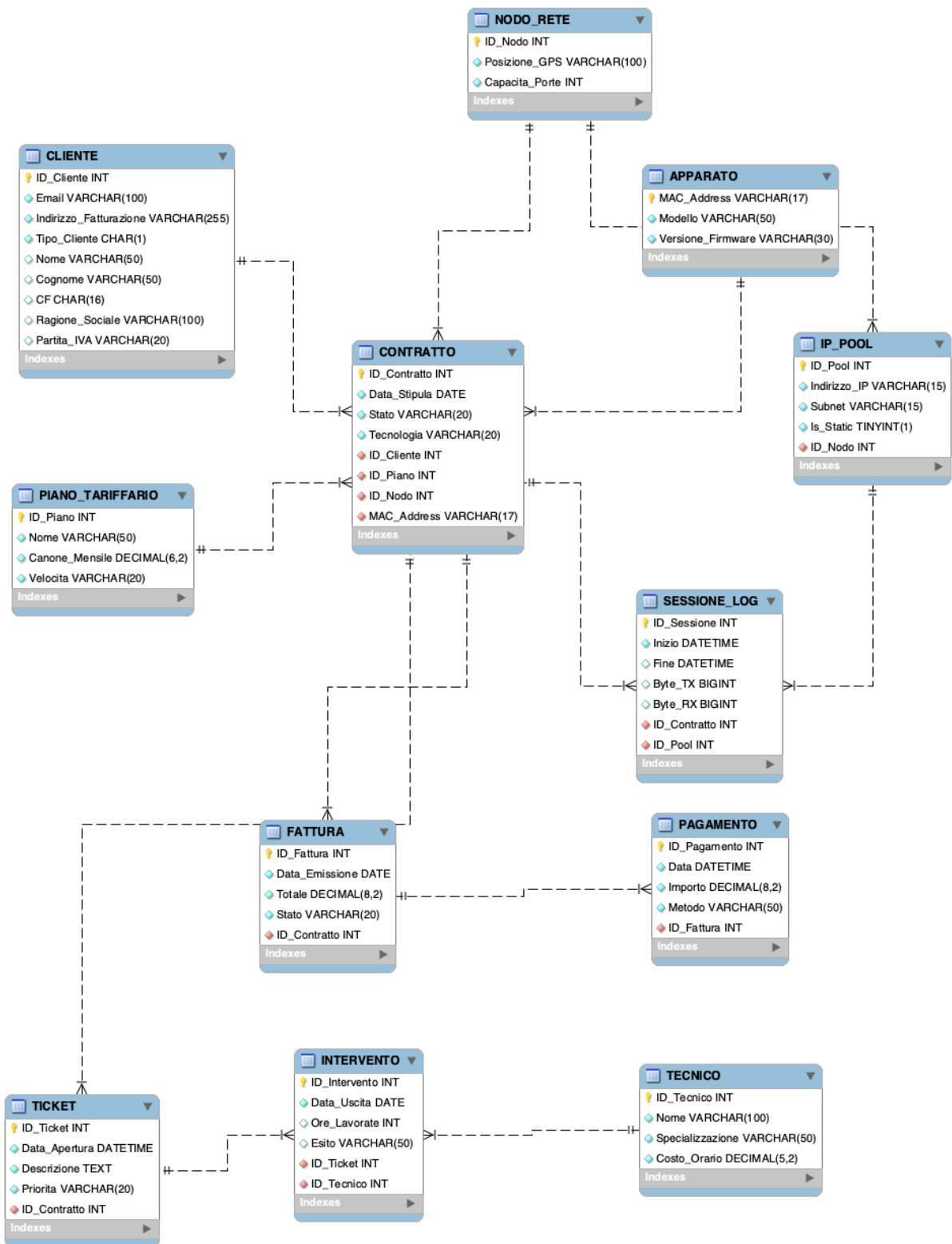


Figura 5: Schema EER generato tramite Reverse Engineering in MySQL Workbench. Sono visibili i tipi di dato, le chiavi primarie, le chiavi esterne e i vincoli di integrità referenziale (rappresentati dalle linee di collegamento).

3 Progettazione Fisica

3.1 Analisi degli Indici e Scelte Prese

Il calcolo si basa sull'architettura fisica del DBMS, assumendo una dimensione standard del blocco disco $B = 4096$ byte. A titolo esemplificativo, analizziamo la tabella `SESSIONE_LOG`, che rappresenta il vero collo di bottiglia del sistema essendo la più sollecitata dal carico di lavoro sia in scrittura (O2) che in lettura legale (O3). Stimando un volume di $N = 730.000.000$ di record e una dimensione media per singola tupla $R = 50$ byte (considerando i vari DATETIME, BIGINT e chiavi esterne), si determina il fattore di blocco dati (bf_r), ovvero il numero di record contenuti in un singolo blocco:

$$bf_r = \lfloor \frac{B}{R} \rfloor = \lfloor \frac{4096}{50} \rfloor = 81 \text{ record/blocco}$$

Da questo si ricava l'occupazione fisica totale della tabella in termini di blocchi (N_B):

$$N_B = \lceil \frac{N}{bf_r} \rceil = \lceil \frac{730.000.000}{81} \rceil = 9.012.346 \text{ blocchi}$$

3.1.1 Confronto dei costi di ricerca

In assenza di indici (Heap File), una ricerca esatta per identificare a chi fosse assegnato un IP in un preciso istante (O3) imporrebbe una scansione sequenziale dell'intera cronologia annuale. Il costo medio per individuare il record si attesterebbe a $N_B/2 \approx 4.506.173$ accessi I/O, un valore inaccettabile che bloccherebbe il server. Implementando invece un Indice Multi-Livello (B+Tree) composito sulla coppia (`ID_Pool`, `Inizio`), il DBMS naviga l'albero logaritmico scendendo dalla radice alle foglie. Per volumi nell'ordine delle centinaia di milioni, la profondità dell'albero richiede circa 4 – 5 I/O per la navigazione dell'indice, più 1 I/O per il recupero del blocco dati, abbattendo il costo totale a soli 5 – 6 accessi I/O.

3.1.2 Analisi dell'indice su `IP_POOL` (`idx_ip_address`)

L'operazione O3 richiede inizialmente di tradurre l'indirizzo IP testuale fornito dalle autorità nel corrispondente `ID_Pool`. Analizziamo la tabella `IP_POOL` con $N = 5.000$ record:

- Dimensione record (R): circa 40 byte (ID, Indirizzo, Subnet, Is_Static, ID_Nodo).
- Fattore di blocco (bf_r): $bf_r = \lfloor \frac{4096}{40} \rfloor = 102 \text{ record / blocco}$.

- Occupazione fisica (N_B): $N_B = \lceil \frac{5000}{102} \rceil = 50$ blocchi.

Nonostante l'attuale dimensione ridotta della tabella porti a un costo di scansione sequenziale modesto ($N_B/2 = 25$ I/O), l'implementazione del B+Tree su `Indirizzo_IP` è una scelta obbligata di scalabilità. In una configurazione "Point Query", l'indice permette di accedere direttamente al record in 2 – 3 I/O. Questa scelta garantirà prestazioni costanti anche quando l'ISP effettuerà la transizione a IPv6, moltiplicando esponenzialmente il numero di subnet e pool gestiti.

3.1.3 Analisi dell'indice su FATTURA (`idx_fattura_stato_data`)

L'operazione O7 (Sospensione contratti morosi) utilizza lo stato del pagamento e la data di emissione come filtri per la ricerca delle fatture insolute da oltre 60 giorni. Analizziamo la tabella `FATTURA` con $N = 7.200.000$ record:

- Dimensione record (R): circa 30 byte (ID, Data, Totale, Stato, ID_Contratto).
- Fattore di blocco (bf_r): $bf_r = \lfloor \frac{4096}{30} \rfloor = 136$ record / blocco.
- Occupazione fisica (N_B): $N_B = \lceil \frac{7.200.000}{136} \rceil = 52.942$ blocchi.

Una scansione sequenziale richiederebbe oltre 26.000 I/O giornalieri per il batch di sospensione. L'indice B+Tree composito su (`Stato`, `Data_Emissione`) è ampiamente giustificato dalla natura dell'operazione Range Query (`WHERE Stato = 'Da Pagare' AND Data_Emissione <= X`). L'indice mantiene i valori ordinati nelle foglie, permettendo al motore di ricerca di isolare istantaneamente solo lo spicchio di fatture critiche senza processare i milioni di record già saldati.

3.2 Implementazione

Sulla base dell'analisi dei costi prestazionali, sono stati definiti in DDL i seguenti indici secondari per ottimizzare le interrogazioni più pesanti:

```

1  -- Ottimizzazione identificazione legale e tracciamento IP (03)
2  CREATE INDEX idx_sessione_pool_inizio ON SESSIONE_LOG(ID_Pool,
3      Inizio);
4
5  -- Velocizzazione traduzione IP logico in ID di sistema (03)
6  CREATE UNIQUE INDEX idx_ip_address ON IP_POOL(Indirizzo_IP);
7
8  -- Ottimizzazione batch notturno sospensione morosi (07)
9  CREATE INDEX idx_fattura_stato_data ON FATTURA(Stato,
10     Data_Emissione);

```

3.2.1 Valutazione dell'overhead di scrittura

Nonostante il volume mastodontico della tabella `SESSIONE_LOG` (730 milioni di record annui), si è scelto deliberatamente di **non** inserire ulteriori indici secondari (ad esempio sui volumi di traffico `Byte_TX` o `Byte_RX` per fini statistici). Poiché l'operazione O2 (registrazione massiva dei log) riversa nel database circa 2.000.000 di record al giorno con un'altissima frequenza di *INSERT*, l'aggiunta di indici non strettamente necessari introdurrebbe un overhead di scrittura catastrofico. Ogni inserimento comporterebbe infatti il blocco della pagina fisica su disco e il continuo ribilanciamento dei nodi dell'albero B+Tree. Si è quindi scelto di limitare l'indicizzazione al solo indispensabile parametro legale (`ID_Pool`, `Inizio`), privilegiando in modo assoluto la velocità di scrittura (throughput) nella fase critica del tracciamento di rete.

3.3 Normalizzazione (3NF e BCNF)

La normalizzazione è il processo di organizzazione dei dati nel database volto a minimizzare le ridondanze strutturali e prevenire le anomalie di inserimento, aggiornamento e cancellazione. Lo schema relazionale prodotto per l'ISP è stato analizzato rigorosamente seguendo la gerarchia delle forme normali.

3.3.1 Terza Forma Normale (3NF)

Una relazione si trova in Terza Forma Normale se è già in 2NF e ogni attributo non primo dipende dalla chiave primaria in modo diretto, ovvero senza dipendenze transitive. Nello schema progettato, la 3NF è pienamente rispettata poiché si verificano le seguenti condizioni:

- **Assenza di dipendenze parziali:** L'adozione sistematica di chiavi primarie semplici e surrogate (es. `ID_Contratto`, `ID_Sessione`) o atomiche naturali (es. `MAC_Address` per la tabella `APPARATO`) rende strutturalmente impossibile che un attributo dipenda solo da una parte della chiave.
- **Assenza di dipendenze transitive:** Non esistono attributi non chiave che ne determinano altri. Ad esempio, nella tabella `CLIENTE`, le informazioni come l'`Email`, il Codice Fiscale o la Partita IVA dipendono esclusivamente dall'identificativo univoco `ID_Cliente` e non si determinano funzionalmente a vicenda.

3.3.2 Forma Normale di Boyce-Codd (BCNF)

La BCNF rappresenta un'estensione più restrittiva della 3NF. Una relazione è in BCNF se, per ogni dipendenza funzionale non banale $X \rightarrow Y$, il determinante X è una superchiave. Di seguito si riporta la dimostrazione per le entità cardine del sistema ISP:

- **Tabella IP_POOL:** Questa relazione presenta due chiavi candidate: la chiave primaria `ID_Pool` e l'attributo `Indirizzo_IP` (sottoposto a vincolo `UNIQUE` per evitare duplicazioni di routing). Poiché tutte le dipendenze funzionali all'interno della tabella (es. `Subnet`, `Is_Static`) hanno come determinante una di queste due chiavi, e ambedue sono superchiavi, la tabella è in perfetta BCNF.
- **Tabella CONTRATTO:** La chiave primaria `ID_Contratto` determina tutti i campi descrittivi (`Stato`, `Data`) e i riferimenti esterni. Inoltre, la relazione 1:1 con l'infrastruttura rende il `MAC_Address` (chiave esterna vincolata come `UNIQUE`) una chiave candidata alternativa. Essendo gli unici determinanti della tabella coincidenti con superchiavi, non si generano anomalie.
- **Tabella PIANO_TARIFFARIO:** Gli attributi commerciali come `Canone_Mensile` e `Velocita` dipendono funzionalmente e unicamente dalla chiave primaria semplice `ID_Piano`. Poiché l'unico determinante presente è la chiave primaria stessa, la relazione rispetta rigorosamente la BCNF.

L'intero schema relazionale, essendo privo di dipendenze funzionali in cui il determinante non sia una superchiave, assicura la massima integrità del dominio dati e la totale assenza di ridondanze non pianificate.

3.4 DDL completo

Di seguito viene riportato lo script SQL contenente il *Data Definition Language* (DDL) utilizzato per generare il database dell'ISP. Le istruzioni sono ordinate gerarchicamente per evitare conflitti con i vincoli di integrità referenziale durante la fase di istanziazione.

```
1  -- 1. Creazione Tabelle Indipendenti (Entita Principali)
2
3  CREATE TABLE CLIENTE (
4      ID_Cliente INT AUTO_INCREMENT PRIMARY KEY,
5      Email VARCHAR(100) NOT NULL,
6      Indirizzo_Fatturazione VARCHAR(255) NOT NULL,
7      Tipo_Cliente CHAR(1) NOT NULL,
8      Nome VARCHAR(50),
9      Cognome VARCHAR(50),
10     CF CHAR(16),
11     Ragione_Sociale VARCHAR(100),
12     Partita_IVA VARCHAR(20)
13 );
14
15 CREATE TABLE PIANO_TARIFFARIO (
16     ID_Piano INT AUTO_INCREMENT PRIMARY KEY,
17     Nome VARCHAR(50) NOT NULL,
18     Canone_Mensile DECIMAL(6,2) NOT NULL,
19     Velocita VARCHAR(20) NOT NULL
20 );
21
22 CREATE TABLE NODO_RETE (
23     ID_Nodo INT AUTO_INCREMENT PRIMARY KEY,
24     Posizione_GPS VARCHAR(100) NOT NULL,
25     Capacita_Porte INT NOT NULL
26 );
27
28 CREATE TABLE APPARATO (
29     MAC_Address VARCHAR(17) PRIMARY KEY,
30     Modello VARCHAR(50) NOT NULL,
31     Versione_Firmware VARCHAR(30) NOT NULL
32 );
33
34 CREATE TABLE TECNICO (
35     ID_Tecnico INT AUTO_INCREMENT PRIMARY KEY,
```

```

36     Nome VARCHAR(100) NOT NULL ,
37     Specializzazione VARCHAR(50) NOT NULL ,
38     Costo_Orario DECIMAL(5,2) NOT NULL
39 );
40
41 -- 2. Creazione Tabelle Dipendenti (Livello 1)
42
43 CREATE TABLE IP_POOL (
44     ID_Pool INT AUTO_INCREMENT PRIMARY KEY ,
45     Indirizzo_IP VARCHAR(15) NOT NULL UNIQUE ,
46     Subnet VARCHAR(15) NOT NULL ,
47     Is_Static BOOLEAN NOT NULL DEFAULT FALSE ,
48     ID_Nodo INT NOT NULL ,
49     FOREIGN KEY (ID_Nodo) REFERENCES NODO_RETE(ID_Nodo) ON DELETE
        RESTRICT
50 );
51
52 CREATE TABLE CONTRATTO (
53     ID_Contratto INT AUTO_INCREMENT PRIMARY KEY ,
54     Data_Stipula DATE NOT NULL ,
55     Stato VARCHAR(20) NOT NULL DEFAULT 'Attivo' ,
56     Tecnologia VARCHAR(20) NOT NULL ,
57     ID_Cliente INT NOT NULL ,
58     ID_Piano INT NOT NULL ,
59     ID_Nodo INT NOT NULL ,
60     MAC_Address VARCHAR(17) NOT NULL UNIQUE ,
61     FOREIGN KEY (ID_Cliente) REFERENCES CLIENTE(ID_Cliente) ON
        DELETE CASCADE ,
62     FOREIGN KEY (ID_Piano) REFERENCES PIANO_TARIFFARIO(ID_Piano) ON
        DELETE RESTRICT ,
63     FOREIGN KEY (ID_Nodo) REFERENCES NODO_RETE(ID_Nodo) ON DELETE
        RESTRICT ,
64     FOREIGN KEY (MAC_Address) REFERENCES APPARATO(MAC_Address) ON
        DELETE RESTRICT
65 );
66
67 -- 3. Creazione Tabelle Dipendenti (Livello 2)
68
69 CREATE TABLE SESSIONE_LOG (
70     ID_Sessione INT AUTO_INCREMENT PRIMARY KEY ,

```

```

71     Inizio DATETIME NOT NULL,
72     Fine DATETIME,
73     Byte_TX BIGINT DEFAULT 0,
74     Byte_RX BIGINT DEFAULT 0,
75     ID_Contratto INT NOT NULL,
76     ID_Pool INT NOT NULL,
77     FOREIGN KEY (ID_Contratto) REFERENCES CONTRATTO(ID_Contratto)
78         ON DELETE CASCADE,
79     FOREIGN KEY (ID_Pool) REFERENCES IP_POOL(ID_Pool) ON DELETE
80         RESTRICT
81 );
82
83 CREATE TABLE TICKET (
84     ID_Ticket INT AUTO_INCREMENT PRIMARY KEY,
85     Data_Apertura DATETIME NOT NULL,
86     Descrizione TEXT NOT NULL,
87     Priorita VARCHAR(20) NOT NULL,
88     ID_Contratto INT NOT NULL,
89     FOREIGN KEY (ID_Contratto) REFERENCES CONTRATTO(ID_Contratto)
90         ON DELETE CASCADE
91 );
92
93 CREATE TABLE FATTURA (
94     ID_Fattura INT AUTO_INCREMENT PRIMARY KEY,
95     Data_Emissione DATE NOT NULL,
96     Totale DECIMAL(8,2) NOT NULL,
97     Stato VARCHAR(20) NOT NULL DEFAULT 'Da Pagare',
98     ID_Contratto INT NOT NULL,
99     FOREIGN KEY (ID_Contratto) REFERENCES CONTRATTO(ID_Contratto)
100         ON DELETE CASCADE
101 );
102
103 -- 4. Creazione Tabelle Dipendenti (Livello 3)
104
105 CREATE TABLE INTERVENTO (
106     ID_Intervento INT AUTO_INCREMENT PRIMARY KEY,
107     Data_Uscita DATE NOT NULL,
108     Ore_Lavorate INT DEFAULT 1,
109     Esito VARCHAR(50),
110     ID_Ticket INT NOT NULL,

```

```

107     ID_Tecnico INT NOT NULL,
108     FOREIGN KEY (ID_Ticket) REFERENCES TICKET(ID_Ticket) ON DELETE
        CASCADE,
109     FOREIGN KEY (ID_Tecnico) REFERENCES TECNICO(ID_Tecnico) ON
        DELETE RESTRICT
110 );
111
112 CREATE TABLE PAGAMENTO (
113     ID_Pagamento INT AUTO_INCREMENT PRIMARY KEY,
114     Data DATETIME NOT NULL,
115     Importo DECIMAL(8,2) NOT NULL,
116     Metodo VARCHAR(50) NOT NULL,
117     ID_Fattura INT NOT NULL,
118     FOREIGN KEY (ID_Fattura) REFERENCES FATTURA(ID_Fattura) ON
        DELETE CASCADE
119 );

```

3.5 Implementazione della Logica Attiva (Trigger)

A garanzia dell'integrità complessa del database e per automatizzare i processi di business delineati nella progettazione concettuale, sono stati implementati i seguenti costrutti di logica attiva (Trigger) sul motore DBMS MySQL. L'utilizzo della direttiva `DELIMITER` è necessario per permettere al database di compilare il blocco di istruzioni multiple al suo interno.

T1 - Protezione Infrastruttura (BEFORE DELETE su NODO_RETE)

Questo trigger impedisce l'eliminazione accidentale (o dolosa) di un nodo fisico (PoP) qualora esso abbia ancora un'importanza strategica per la rete, ovvero se gestisce degli IP Pool attivi o se serve contratti attualmente in stato "Attivo". Invece di affidarsi al semplice vincolo `RESTRICT`, fornisce un messaggio di errore parlante all'operatore.

```
1 DELIMITER //
2 CREATE TRIGGER T1_Protezione_Infrastruttura
3 BEFORE DELETE ON NODO_RETE
4 FOR EACH ROW
5 BEGIN
6     DECLARE v_contratti_attivi INT;
7     DECLARE v_pool_associati INT;
8
9     SELECT COUNT(*) INTO v_contratti_attivi FROM CONTRATTO
10    WHERE ID_Nodo = OLD.ID_Nodo AND Stato = 'Attivo';
11
12    SELECT COUNT(*) INTO v_pool_associati FROM IP_POOL
13    WHERE ID_Nodo = OLD.ID_Nodo;
14
15    IF v_contratti_attivi > 0 OR v_pool_associati > 0 THEN
16        SIGNAL SQLSTATE '45000'
17        SET MESSAGE_TEXT = 'Impossibile eliminare: il Nodo gestisce
18                                ancora risorse o contratti attivi.';
19    END IF;
20 END; //
21 DELIMITER ;
```

T2 - Chiusura Automatica Ticket (AFTER INSERT su INTERVENTO)

Automatizza il workflow dell'assistenza tecnica. Quando un tecnico compila il reportino di un intervento assegnandogli un esito "Positivo", il sistema chiude automaticamente il

Ticket padre, riducendo il carico di data entry per il personale del call center. (Nota: si assume l'aggiunta logica di un attributo 'Stato' al Ticket durante l'evoluzione del DB).

```
1 DELIMITER //
```

```
2 CREATE TRIGGER T2_Chiusura_Automatica_Ticket
```

```
3 AFTER INSERT ON INTERVENTO
```

```
4 FOR EACH ROW
```

```
5 BEGIN
```

```
6     IF NEW.Esito = 'Positivo' THEN
```

```
7         -- Aggiorna lo stato del Ticket (ipotizzando il campo Stato
```

```
8         nella tabella)
```

```
9         UPDATE TICKET
```

```
10        SET Priorita = 'Risolto' -- o Stato = 'Chiuso'
```

```
11        WHERE ID_Ticket = NEW.ID_Ticket;
```

```
12    END IF;
```

```
13 END; //
```

```
14 DELIMITER ;
```

T3 - Riconciliazione Contabile (AFTER INSERT su PAGAMENTO)

Gestisce i pagamenti rateali o parziali. Ad ogni nuova transazione economica, il trigger somma tutti i pagamenti effettuati per quella specifica fattura. Se il totale versato raggiunge o supera l'importo dovuto, la fattura transita automaticamente nello stato "Saldato".

```
1 DELIMITER //
```

```
2 CREATE TRIGGER T3_Riconciliazione_Contabile
```

```
3 AFTER INSERT ON PAGAMENTO
```

```
4 FOR EACH ROW
```

```
5 BEGIN
```

```
6     DECLARE v_totale_pagato DECIMAL(8,2);
```

```
7     DECLARE v_totale_fattura DECIMAL(8,2);
```

```
8
```

```
9     -- Calcola quanto e' stato pagato finora per questa fattura
```

```
10    SELECT SUM(Importo) INTO v_totale_pagato FROM PAGAMENTO
```

```
11    WHERE ID_Fattura = NEW.ID_Fattura;
```

```
12
```

```
13    -- Recupera il totale dovuto
```

```
14    SELECT Totale INTO v_totale_fattura FROM FATTURA
```

```
15    WHERE ID_Fattura = NEW.ID_Fattura;
```

```
16
```

```

17      -- Se l'importo copre la fattura, cambiane lo stato
18      IF IFNULL(v_totale_pagato, 0) >= v_totale_fattura THEN
19          UPDATE FATTURA
20          SET Stato = 'Saldato'
21          WHERE ID_Fattura = NEW.ID_Fattura;
22      END IF;
23  END; //
24  DELIMITER ;

```

T4 - Blocco Clienti Morosi (BEFORE INSERT su CONTRATTO)

Rappresenta una regola di business critica: il sistema intercetta e blocca la creazione di un nuovo contratto se l'identificativo del Cliente è associato a fatture insolute da oltre 90 giorni (Clienti "Bad Payer").

```

1  DELIMITER //
2  CREATE TRIGGER T4_Blocco_Clienti_Morosi
3  BEFORE INSERT ON CONTRATTO
4  FOR EACH ROW
5  BEGIN
6      DECLARE v_fatture_insolute INT;
7
8      SELECT COUNT(*) INTO v_fatture_insolute
9      FROM FATTURA f
10     JOIN CONTRATTO c ON f.ID_Contratto = c.ID_Contratto
11     WHERE c.ID_Cliente = NEW.ID_Cliente
12           AND f.Stato != 'Saldato'
13           AND DATEDIFF(CURRENT_DATE(), f.Data_Emissione) > 90;
14
15     IF v_fatture_insolute > 0 THEN
16         SIGNAL SQLSTATE '45000'
17         SET MESSAGE_TEXT = 'Operazione negata: il cliente risulta
18                               moroso da oltre 90 giorni.';
19     END IF;
20 END; //
21 DELIMITER ;

```

T5 - Unicità Apparato Fisico (BEFORE INSERT su CONTRATTO)

Oltre al vincolo UNIQUE già presente a livello di DDL, questo trigger implementa un

controllo logico più granulare: verifica preventivamente che l'Apparato (es. l'ONT della fibra o il Router) non sia correntemente associato a un altro contratto con stato "Attivo", prevenendo duplicazioni di MAC Address sulla rete viva.

```
1 DELIMITER //
2 CREATE TRIGGER T5_Unicita_Apparato_Fisico
3 BEFORE INSERT ON CONTRATTO
4 FOR EACH ROW
5 BEGIN
6     DECLARE v_mac_in_uso INT;
7
8     SELECT COUNT(*) INTO v_mac_in_uso FROM CONTRATTO
9     WHERE MAC_Address = NEW.MAC_Address
10        AND Stato = 'Attivo';
11
12     IF v_mac_in_uso > 0 THEN
13         SIGNAL SQLSTATE '45000'
14         SET MESSAGE_TEXT = 'Anomalia Hardware: MAC Address gia
15                             assegnato a una linea attiva.';
16     END IF;
17 END; //
18 DELIMITER ;
```

T6 - Prevenzione Conflitti IP (BEFORE INSERT su SESSIONE_LOG)

Essenziale per la validità legale dei log. Assicura che un singolo Indirizzo IP logico (rappresentato dall'ID_Pool) non venga assegnato a una nuova sessione di navigazione se esiste già un'altra sessione aperta (con Fine IS NULL) che sta utilizzando lo stesso IP, evitando l'accavallamento dei tracciamenti.

```
1 DELIMITER //
2 CREATE TRIGGER T6_Prevenzione_Conflitti_IP
3 BEFORE INSERT ON SESSIONE_LOG
4 FOR EACH ROW
5 BEGIN
6     DECLARE v_sessioni_attive INT;
7
8     SELECT COUNT(*) INTO v_sessioni_attive FROM SESSIONE_LOG
9     WHERE ID_Pool = NEW.ID_Pool
10        AND Fine IS NULL;
11
```

```

12     IF v_sessioni_attive > 0 THEN
13         SIGNAL SQLSTATE '45000'
14         SET MESSAGE_TEXT = 'Errore di Routing: Indirizzo IP
                                attualmente in uso da un''altra sessione.';
15     END IF;
16 END; //
17 DELIMITER ;

```

T7 - Validazione Temporale Logica (BEFORE INSERT su CONTRATTO)

Garantisce la coerenza temporale in fase di data entry, bloccando alla radice l'inserimento di un contratto la cui Data_Stipula sia indicata nel futuro rispetto al timestamp corrente del server database.

```

1 DELIMITER //
2 CREATE TRIGGER T7_Validazione_Temporale
3 BEFORE INSERT ON CONTRATTO
4 FOR EACH ROW
5 BEGIN
6     IF NEW.Data_Stipula > CURRENT_DATE() THEN
7         SIGNAL SQLSTATE '45000'
8         SET MESSAGE_TEXT = 'Incoerenza Temporale: La data di
                                stipula non puo essere nel futuro.';
9     END IF;
10 END; //
11 DELIMITER ;

```

3.6 Query SQL per le operazioni principali

Di seguito vengono presentate le query SQL relative alle sette operazioni principali (DML) identificate durante la fase di analisi dei requisiti. Ogni istruzione è accompagnata da una breve descrizione della logica di business implementata.

O1 - Inserimento nuovo contratto con assegnazione apparato e IP

Questa query registra un nuovo abbonamento associandolo a un cliente esistente, al piano tariffario scelto, al nodo infrastrutturale di pertinenza e al MAC Address dell'apparato in comodato d'uso. L'inserimento scatena automaticamente i controlli dei trigger T4 (blocco morosi), T5 (unicità apparato) e T7 (coerenza temporale).

```
1 INSERT INTO CONTRATTO (Data_Stipula, Stato, Tecnologia, ID_Cliente,
   ID_Piano, ID_Nodo, MAC_Address)
2 VALUES (CURRENT_DATE(), 'Attivo', 'FTTH', 105, 2, 14, '00:1A:2B:3C
   :4D:5E');
```

O2 - Registrazione massiva log di sessione (alta frequenza)

Operazione ad altissima frequenza. Inserisce i dati di telemetria e navigazione di un utente al termine della sua sessione. In fase di apertura della sessione, scatterà il trigger T6 per prevenire conflitti sull'indirizzo IP.

```
1 INSERT INTO SESSIONE_LOG (Inizio, Fine, Byte_TX, Byte_RX,
   ID_Contratto, ID_Pool)
2 VALUES ('2026-05-01 08:15:00', '2026-05-01 12:45:00', 1540000000,
   8500000000, 105, 42);
```

O3 - Identificazione cliente tramite IP e timestamp (uso legale)

Interrogazione critica richiesta dalle autorità competenti (Polizia Postale). Partendo da un Indirizzo IP logico e un preciso istante temporale, il sistema effettua una serie di JOIN per risalire esattamente all'anagrafica della persona fisica o giuridica che stava utilizzando quell'IP in quel momento.

```
1 SELECT c.Nome, c.Cognome, c.Ragione_Sociale, c.CF, c.Partita_IVA, s
   .Inizio, s.Fine
2 FROM SESSIONE_LOG s
3 JOIN CONTRATTO co ON s.ID_Contratto = co.ID_Contratto
4 JOIN CLIENTE c ON co.ID_Cliente = c.ID_Cliente
5 JOIN IP_POOL p ON s.ID_Pool = p.ID_Pool
6 WHERE p.Indirizzo_IP = '192.168.1.50'
```

```

7      AND '2026-04-15 10:30:00' BETWEEN s.Inizio AND IFNULL(s.Fine,
      CURRENT_TIMESTAMP);

```

O4 - Registrazione intervento tecnico e chiusura ticket

Registra il lavoro svolto da un tecnico sul campo in risposta a un guasto. Essendo l'esito inserito come "Positivo", il motore database invocherà in modo silente il trigger T2, che provvederà ad aggiornare automaticamente lo stato del Ticket originario chiudendolo.

```

1  INSERT INTO INTERVENTO (Data_Uscita, Ore_Lavorate, Esito, ID_Ticket
    , ID_Tecnico)
2  VALUES (CURRENT_DATE(), 2, 'Positivo', 505, 12);

```

O5 - Generazione fatture mensili per contratti attivi

Query di tipo *Batch* eseguita tipicamente al primo giorno del mese. Utilizza la sintassi INSERT INTO ... SELECT per scorrere massivamente tutti i contratti in stato "Attivo", incrociarli con il loro listino e generare in un solo colpo tutte le fatture leggendo il canone dal piano tariffario.

```

1  INSERT INTO FATTURA (Data_Emissione, Totale, Stato, ID_Contratto)
2  SELECT CURRENT_DATE(), p.Canone_Mensile, 'Da Pagare', c.
    ID_Contratto
3  FROM CONTRATTO c
4  JOIN PIANO_TARIFFARIO p ON c.ID_Piano = p.ID_Piano
5  WHERE c.Stato = 'Attivo';

```

O6 - Registrazione pagamento e aggiornamento stato fattura

Registra un versamento da parte del cliente. Non vi è necessità di fare aggiornamenti manuali sulla fattura, poiché l'inserimento di questo record azionerà il trigger T3, il quale ricalcolerà il saldo totale e imposterà la fattura come "Saldato" se l'importo è sufficiente.

```

1  INSERT INTO PAGAMENTO (Data, Importo, Metodo, ID_Fattura)
2  VALUES (CURRENT_TIMESTAMP, 29.90, 'Addebito SEPA', 10024);

```

O7 - Sospensione contratti con fatture insolute da 60 giorni

Script di manutenzione automatizzata. Identifica le utenze con fatture scadute e insolute da oltre 60 giorni e agisce aggiornando direttamente l'entità padre, mettendo in sospensione tecnica l'abbonamento per interrompere l'erogazione del servizio (blocco logico sulla rete).

```
1 UPDATE CONTRATTO
2 SET Stato = 'Sospeso'
3 WHERE ID_Contratto IN (
4     SELECT ID_Contratto
5     FROM FATTURA
6     WHERE Stato = 'Da Pagare'
7         AND DATEDIFF(CURRENT_DATE(), Data_Emissione) > 60
8 );
```

3.7 Dump del database popolato con dati significativi

Al fine di validare la correttezza strutturale dello schema e testare il comportamento delle interrogazioni e dei trigger implementati, il database è stato popolato con un set di dati fittizi ma altamente significativi per il dominio dell'ISP. L'ordine di inserimento rispetta rigorosamente i vincoli gerarchici imposti dalle FOREIGN KEY.

```
1  -- 1. Popolamento Entita Forti (Anagrafiche e Infrastruttura)
2
3  INSERT INTO CLIENTE (Email, Indirizzo_Fatturazione, Tipo_Cliente,
4      Nome, Cognome, CF, Ragione_Sociale, Partita_IVA) VALUES
5      ('mario.rossi@email.it', 'Via Roma 10, Milano', 'P', 'Mario', '
6      Rossi', 'RSSMRA80A01F205X', NULL, NULL),
7      ('amministrazione@techsrl.com', 'Viale Europa 45, Roma', 'A', NULL,
8      NULL, NULL, 'Tech Solutions S.r.l.', 'IT12345678901');
9
10
11 INSERT INTO PIANO_TARIFFARIO (Nome, Canone_Mensile, Velocita)
12     VALUES
13     ('UltraFibra 1000', 29.90, '1000/300 Mbps'),
14     ('Business Pro FWA', 49.90, '200/50 Mbps');
15
16
17 INSERT INTO NODO_RETE (Posizione_GPS, Capacita_Porte) VALUES
18     ('45.4642, 9.1900 - POP Milano Centro', 1024),
19     ('41.9028, 12.4964 - POP Roma EUR', 2048);
20
21
22 INSERT INTO APPARATO (MAC_Address, Modello, Versione_Firmware)
23     VALUES
24     ('00:1A:2B:3C:4D:5E', 'Fritz!Box 7590 AX', 'v7.50'),
25     ('AA:BB:CC:DD:EE:FF', 'Huawei B818 Router', 'v11.0.1');
26
27
28 INSERT INTO TECNICO (Nome, Specializzazione, Costo_Orario) VALUES
29     ('Luca Bianchi', 'Giuntista Fibra Ottica', 35.00),
30     ('Andrea Verdi', 'Tecnico Radio FWA', 30.00);
31
32
33 -- 2. Popolamento Livello 1 (Rete Logica e Contratti)
34
35 INSERT INTO IP_POOL (Indirizzo_IP, Subnet, Is_Static, ID_Nodo)
36     VALUES
37     ('2.34.56.78', '255.255.255.255', FALSE, 1),
38     ('81.20.30.40', '255.255.255.248', TRUE, 2);
39
```

```

29 INSERT INTO CONTRATTO (Data_Stipula, Stato, Tecnologia, ID_Cliente,
    ID_Piano, ID_Nodo, MAC_Address) VALUES
30 ('2025-10-15', 'Attivo', 'FTTH', 1, 1, 1, '00:1A:2B:3C:4D:5E'),
31 ('2025-11-20', 'Attivo', 'FWA', 2, 2, 2, 'AA:BB:CC:DD:EE:FF');
32
33 -- 3. Popolamento Livello 2 (Traffico, Assistenza e Fatturazione)
34
35 INSERT INTO SESSIONE_LOG (Inizio, Fine, Byte_TX, Byte_RX,
    ID_Contratto, ID_Pool) VALUES
36 ('2026-05-01 08:00:00', '2026-05-01 20:00:00', 500000000,
    2500000000, 1, 1),
37 ('2026-05-01 09:00:00', NULL, 1200000000, 8000000000, 2, 2); --
    Sessione ancora aperta
38
39 INSERT INTO TICKET (Data_Apertura, Descrizione, Priorita,
    ID_Contratto) VALUES
40 ('2026-05-02 10:30:00', 'Cadute di linea frequenti', 'Alta', 1);
41
42 INSERT INTO FATTURA (Data_Emissione, Totale, Stato, ID_Contratto)
    VALUES
43 ('2026-05-01', 29.90, 'Da Pagare', 1),
44 ('2026-05-01', 49.90, 'Da Pagare', 2);
45
46 -- 4. Popolamento Livello 3 (Interventi Tecnici e Pagamenti)
47
48 INSERT INTO INTERVENTO (Data_Uscita, Ore_Lavorate, Esito, ID_Ticket
    , ID_Tecnico) VALUES
49 ('2026-05-03', 2, 'Positivo', 1, 1);
50
51 INSERT INTO PAGAMENTO (Data, Importo, Metodo, ID_Fattura) VALUES
52 ('2026-05-04 15:00:00', 29.90, 'Carta di Credito', 1);

```

4 Utilizzo dell'Intelligenza Artificiale

La realizzazione di un sistema informativo completo per un ISP, dalla sua concettualizzazione fino all'implementazione fisica, ha richiesto un notevole sforzo progettuale. Durante lo sviluppo di questo elaborato, l'Intelligenza Artificiale (nello specifico, *Gemini*) è stata impiegata esclusivamente come strumento di supporto tecnico e di *tutoring*, ottimizzando i tempi sulle attività ripetitive e lasciando al sottoscritto la totale paternità delle scelte architetturali, logiche e di dominio.

Nello specifico, l'IA è stata utilizzata nei seguenti ambiti limitati:

- **Revisione della sintassi SQL:** Una volta formulata la logica di business dei Trigger e delle interrogazioni complesse, l'IA è stata utile per identificare tempestivamente piccoli refusi di sintassi (es. la gestione dei `DELIMITER` o le dichiarazioni dei `DECLARE`) tipici del dialetto MySQL.
- **Generazione di dati *dummy*:** Per testare l'effettivo funzionamento del database, l'IA ha fornito supporto nella rapida generazione di script di popolamento (Data Dump) verosimili, automatizzando la scrittura massiva delle clausole `INSERT` su decine di record.
- **Supporto all'impaginazione in $\text{\LaTeX}$:** La stesura della presente relazione ha richiesto formattazioni grafiche avanzate. L'assistente virtuale ha fornito supporto nell'uso corretto di pacchetti specifici su Overleaf (come `listings` per l'evidenziazione del codice SQL e `ulem` per la sottolineatura tratteggiata delle chiavi esterne), migliorando la resa estetica finale del documento senza intervenire sui contenuti accademici redatti.